

CSE312 – Operating Systems

Quiz 9 Hazırlık Soru Seti

Virtual Memory – Address Translation & Kavramsal

Kapsam (asistan mesajına göre):

1. Address translation hesabı (Implementing Virtual Memory)
 2. Virtual memory kavramsal soru
 3. Quiz 8’de zorlanan soruya benzer “ikinci şans” sorusu
-

Gebze Teknik Üniversitesi
Bilgisayar Mühendisliği Bölümü
Spring 2026

Bu soru seti 3 kategoriden oluşmaktadır:

- A) Address Translation – 5 ayrı hesap sorusu
- B) Page Replacement (FIFO / LRU / Optimal) – 2 soru
- C) Kavramsal Virtual Memory – 2 soru

Her sorunun hemen altında ayrıntılı çözüm yer almaktadır.

Contents

1	Kategori A: Address Translation Hesabı (5 Soru)	2
1.1	Soru A1 – Temel Düzeyde Address Translation	2
1.2	Soru A2 – Page Size 512 Bytes, Hex Address	3
1.3	Soru A3 – TLB Hit/Miss Senaryosu	4
1.4	Soru A4 – Karmaşık: Birden Fazla Bilgi Verilen Sistem	5
1.5	Soru A5 – Tersine Giden Hesap: Physical → Virtual	7
2	Kategori B: Page Replacement – “İkinci Şans” Sorusu	8
2.1	Soru B1 – FIFO + LRU + Optimal (3 Frame)	8
2.2	Soru B2 – Farklı String ile FIFO + LRU (4 Frame)	10
3	Kategori C: Kavramsal Virtual Memory Soruları	11
3.1	Soru C1 – Thrashing, Working Set, PFF	11
3.2	Soru C2 – TLB, Page Fault Mekanizması, Karşılaştırma	12
4	Hızlı Başvuru: Özet Tablo	13

1 Kategori A: Address Translation Hesabı (5 Soru)

Genel formül hatırlatması:

Bir virtual address şu iki parçaya bölünür:

$$\text{Virtual Address} = \underbrace{\text{Page Number}}_{\text{üst bitler}} \parallel \underbrace{\text{Offset}}_{\text{alt bitler}}$$

Page size = 2^k byte ise k bit offset, kalan bitler page number'dir.

Page table'dan: **Frame Number** okunur.

Physical address:

$$\text{Physical Address} = (\text{Frame Number} \times \text{Page Size}) + \text{Offset}$$

Adım adım:

1. Page size'dan offset bit sayısını bul: $k = \log_2(\text{page size})$
2. Virtual address'i binary'e çevir
3. Üst bitleri page number, alt k biti offset olarak ayır
4. Page table'dan frame number'i bul
5. Physical address = frame number bitlerini offset'in önerine koy (veya formül uygula)

1.1 Soru A1 – Temel Düzeyde Address Translation

Soru A1

Bir sistemde page size **256 bytes**'tır. Sayfa tablosu (page table) aşağıdadır:

Page Number	Frame Number
0	5
1	2
2	8
3	1

Aşağıdaki virtual address'leri physical address'e çevirin:

- (a) Virtual address = **0x0041** (decimal: 65)
- (b) Virtual address = **0x0128** (decimal: 296)
- (c) Virtual address = **0x02FF** (decimal: 767)

CEVAP

Ön hazırlık:

Page size = 256 bytes = 2^8 bytes $\Rightarrow$ **offset = 8 bit**

16-bit virtual address'te: üst 8 bit = page number, alt 8 bit = offset

(a) Virtual address = **0x0041 = 65**

Binary: 0000 0000 | 0100 0001

⇒ Page number = 0x00 = 0, Offset = 0x41 = 65

Page table: Page 0 → Frame 5

Physical address = $5 \times 256 + 65 = 1280 + 65 = 1345$ (= 0x0541)

(b) Virtual address = 0x0128 = 296

Binary: 0000 0001 | 0010 1000

⇒ Page number = 0x01 = 1, Offset = 0x28 = 40

Page table: Page 1 → Frame 2

Physical address = $2 \times 256 + 40 = 512 + 40 = 552$ (= 0x0228)

(c) Virtual address = 0x02FF = 767

Binary: 0000 0010 | 1111 1111

⇒ Page number = 0x02 = 2, Offset = 0xFF = 255

Page table: Page 2 → Frame 8

Physical address = $8 \times 256 + 255 = 2048 + 255 = 2303$ (= 0x08FF)

1.2 Soru A2 – Page Size 512 Bytes, Hex Address

Soru A2

Bir sistemde:

- Page size = **512 bytes**
- Virtual address space = **14 bit**
- Physical memory = **8 KB**

Page table:

Page Number	Frame Number	Valid Bit
0	3	1
1	–	0
2	0	1
3	2	1
4	–	0
5	1	1

Aşağıdaki virtual address'ler için: (i) page number ve offset'i bulun, (ii) valid bit'i kontrol edin, (iii) physical address'i hesaplayın ya da page fault olduğunu belirtin.

- Virtual address = **0x0600** (decimal: 1536)
- Virtual address = **0x0A1F** (decimal: 2591)
- Virtual address = **0x12FF** (decimal: 4863)

CEVAP

Ön hazırlık:

Page size = 512 = 2^9 bytes ⇒ **offset = 9 bit**

14-bit virtual address: üst 5 bit = page number, alt 9 bit = offset

Physical memory = 8 KB = 2^{13} bytes; frame sayısı = $8192/512 = 16$, frame number = 4 bit

(a) Virtual address = 0x0600 = 1536

Binary (14 bit): 00 011 | 0 0000 0000

⇒ Page number = 3, Offset = 0

Valid bit = 1, Frame number = 2

Physical address = $2 \times 512 + 0 = 1024$ (= 0x0400)

(b) Virtual address = 0x0A1F = 2591

Binary (14 bit): 00 101 | 0 0001 1111

⇒ Page number = 5, Offset = 31

Valid bit = 1, Frame number = 1

Physical address = $1 \times 512 + 31 = 543$ (= 0x021F)

(c) Virtual address = 0x12FF = 4863

Binary (14 bit): 10 010 | 0 1111 1111

⇒ Page number = 18? – dur, kontrol edelim.

14-bit max = 16383. 4863 binary: 01 001 | 0 1111 1111

⇒ Page number = 01001 = 9 ⇒ Page table'da yok (sadece 0–5 var)

PAGE FAULT! (Geçersiz sayfa numarası – page table'da giriş yok)

	Page #	Offset	Frame #	Physical Addr.
(a)	3	0	2	1024
(b)	5	31	1	543
(c)	9	255	–	PAGE FAULT

1.3 Soru A3 – TLB Hit/Miss Senaryosu

Soru A3

Bir sistemde page size = 1 KB (1024 bytes), virtual address = 16 bit'tir.

TLB (Translation Lookaside Buffer) ve page table durumu:

TLB	
Page #	Frame #
2	6
7	4

Page Table	
Page #	Frame #
0	9
1	3
2	6
3	1
4	7
5	0
6	5
7	4

Aşağıdaki virtual address'ler için: (i) TLB hit mi miss mi belirleyin, (ii) physical address'i hesaplayın.

(a) Virtual address = 0x1C08 (decimal: 7176)

(b) Virtual address = 0x08FF (decimal: 2303)

(c) Virtual address = 0x1403 (decimal: 5123)

CEVAP**Ön hazırlık:**

Page size = 1024 = 2^{10} bytes $\Rightarrow$ **offset = 10 bit**

16-bit address: üst 6 bit = page number, alt 10 bit = offset

(a) Virtual address = 0x1C08 = 7176

Binary: 000111 | 00 0000 1000

$\Rightarrow$ Page number = **7**, Offset = **8**

TLB: Page 7 $\in$ TLB $\Rightarrow$ **TLB HIT**

Frame number = **4**

Physical address = $4 \times 1024 + 8 = 4096 + 8 = \mathbf{4104}$ (= 0x1008)

(b) Virtual address = 0x08FF = 2303

Binary: 000010 | 00 1111 1111

$\Rightarrow$ Page number = **2**, Offset = **255**

TLB: Page 2 $\in$ TLB $\Rightarrow$ **TLB HIT**

Frame number = **6**

Physical address = $6 \times 1024 + 255 = 6144 + 255 = \mathbf{6399}$ (= 0x18FF)

(c) Virtual address = 0x1403 = 5123

Binary: 000101 | 00 0000 0011

$\Rightarrow$ Page number = **5**, Offset = **3**

TLB: Page 5 $\notin$ TLB $\Rightarrow$ **TLB MISS** (page table'a git)

Page table: Page 5 $\rightarrow$ Frame **0**

Physical address = $0 \times 1024 + 3 = \mathbf{3}$ (= 0x0003)

	Page #	Offset	TLB?	Frame #	Physical Addr.
(a)	7	8	HIT	4	4104
(b)	2	255	HIT	6	6399
(c)	5	3	MISS	0	3

TLB Mantığı

TLB hit: Tek bir ek bellek erişimi (TLB). Çok hızlı.

TLB miss: TLB + page table lookup = 2 bellek erişimi (+ frame'deki veri).

TLB, sık kullanılan page-frame eşlemelerini cache'ler. Context switch'te genellikle flush'lanır.

1.4 Soru A4 – Karmaşık: Birden Fazla Bilgi Verilen Sistem**Soru A4**

Bir sistemde:

- **Logical (virtual) address space = 32 pages**
- Page size = **2 KB** (2048 bytes)
- Physical memory = **64 KB**

Page table (kısmen gösterilmiştir; gösterilmeyen sayfalar bellekte değildir):

Page #	Frame #
0	12
1	0
2	5
3	7
4	– (not in memory)
5	2

- (a) Kaç bit virtual address, kaç bit physical address gerekir? Gösteriniz.
 (b) Virtual address = **0x4A05** için physical address'i bulunuz.
 (c) Virtual address = **0x8200** için ne olur?

CEVAP

(a) Bit hesabı:

Virtual address:

- $32 \text{ page} = 2^5 \Rightarrow \text{page number} = 5 \text{ bit}$
- $\text{Page size} = 2048 = 2^{11} \Rightarrow \text{offset} = 11 \text{ bit}$
- Toplam: $5 + 11 = \mathbf{16 \text{ bit}}$ virtual address

Physical address:

- $\text{Physical memory} = 64 \text{ KB} = 2^{16} \text{ bytes}$
- $\text{Frame sayısı} = 65536/2048 = 32 \Rightarrow \text{frame number} = 5 \text{ bit}$
- Offset = 11 bit (aynı)
- Toplam: $5 + 11 = \mathbf{16 \text{ bit}}$ physical address

(b) Virtual address = **0x4A05** = 18949

Binary (16 bit): 01001 | 010 0000 0101

$\Rightarrow \text{Page number} = 01001 = \mathbf{9}$

$\Rightarrow \text{Offset} = 010 \ 0000 \ 0101 = \mathbf{517}$

Page 9 $\Rightarrow$ page table'da yok $\Rightarrow$ **PAGE FAULT!**

Kontrol: $0x4A05 = 18949$. Page size = 2048.

$18949 \div 2048 = 9$ (tam bölüm) $\Rightarrow$ Page 9, Offset = $18949 - 9 \times 2048 = 18949 - 18432 = 517$.

Page 9 tabloda yok $\Rightarrow$ **PAGE FAULT.**

(c) Virtual address = **0x8200** = 33280

16-bit max = 65535. $33280 \leq 65535$, geçerli görünür.

$33280 \div 2048 = 16$ (tam), Offset = 0 $\Rightarrow$ Page 16.

Page 16 tabloda yok $\Rightarrow$ **PAGE FAULT!**

Not

Page table sadece 0–5 arası verilmiş. Gerisi “not in memory” — bu page fault anlamına gelir. Her geçersiz veya bellekte olmayan sayfa erişimi page fault tetikler.

1.5 Soru A5 – Tersine Giden Hesap: Physical → Virtual

Soru A5

Aynı sistemde page size = **512 bytes**, page table aşağıdadır:

Page #	Frame #
0	4
1	7
2	2
3	6
4	0

- (a) Physical address = **1537**'nin hangi virtual address'e karşılık geldiğini bulunuz.
 (b) Physical address = **3584**'ün virtual address'ini bulunuz.
 (c) Physical frame 2'de offset 100'deki byte'a erişmek isteyen sürece hangi virtual address'i kullanması gerekir?

CEVAP

Ön hazırlık:

Page size = 512 = $2^9 \Rightarrow$ offset = 9 bit

(a) Physical address = 1537

Frame number = $\lfloor 1537/512 \rfloor = \lfloor 2.99 \rfloor = 3$

Offset = $1537 - 3 \times 512 = 1537 - 1536 = 1$

Frame 3'ü içeren page: Page table'a ters bak $\Rightarrow$ Frame 3'ü tutan sayfa: **Yok!**

(Verilen tabloda frame 3 yok: sadece 4, 7, 2, 6, 0 var)

Bu physical address hiçbir virtual page'e map edilmemiştir!

(b) Physical address = 3584

Frame number = $\lfloor 3584/512 \rfloor = \lfloor 7 \rfloor = 7$

Offset = $3584 - 7 \times 512 = 3584 - 3584 = 0$

Frame 7'yi tutan page: **Page 1** (tabloya bak: Page 1 $\rightarrow$ Frame 7)

Virtual address = $1 \times 512 + 0 = 512$

(c) Frame 2, offset 100

Physical address = $2 \times 512 + 100 = 1024 + 100 = 1124$

Frame 2'yi tutan page: **Page 2** (Page 2 $\rightarrow$ Frame 2)

Virtual address = $2 \times 512 + 100 = 1124$

(Bu çöküde page number ile frame number aynı olduğu için virtual = physical, ama bu genelde böyle olmaz!)

	Phys. Addr.	Frame #	Page #	Virt. Addr.
(a)	1537	3	yok	–
(b)	3584	7	1	512
(c)	1124	2	2	1124

2 Kategori B: Page Replacement – “İkinci Şans” Sorusu

Quiz 8 Part B’de aynı reference string sorulmuştu (1 2 3 4 1 2 5 1 2 3 4 5, 3 frame). Asistan “ikinci şans” dedi — bu sorunun aynısı veya çok benzeri gelecek. Farklı frame sayıları, farklı stringler veya **Optimal** eklenmesiyle gelebilir.

2.1 Soru B1 – FIFO + LRU + Optimal (3 Frame)

Soru B1 – Quiz 8 Tekrarı + Optimal Eklendi

3 page frame’i olan bir sistemde aşağıdaki reference string verilmiştir:

1 2 3 4 1 2 5 1 2 3 4 5

- (a) **FIFO** algoritmasını uygulayarak toplam page fault sayısını bulunuz.
- (b) **LRU** algoritmasını uygulayarak toplam page fault sayısını bulunuz.
- (c) **Optimal** algoritmasını uygulayarak toplam page fault sayısını bulunuz.
- (d) Hangi algoritma daha iyi performans göstermektedir? Kısaca açıklayın.

CEVAP

Reference string: 1 2 3 4 1 2 5 1 2 3 4 5 (3 frame)

(a) FIFO:

FIFO’da ilk giren ilk çıkar (en eski sayfa gider).

Ref	Frame 1	Frame 2	Frame 3	Fault?
1	1	–	–	F
2	1	2	–	F
3	1	2	3	F
4	4	2	3	F (1 çıktı)
1	4	1	3	F (2 çıktı)
2	4	1	2	F (3 çıktı)
5	5	1	2	F (4 çıktı)
1	5	1	2	– (hit)
2	5	1	2	– (hit)
3	5	3	2	F (1 çıktı)
4	5	3	4	F (2 çıktı)
5	5	3	4	– (hit)

FIFO toplam page fault = 9

(b) LRU:

LRU’da en uzun süredir kullanılmayan sayfa çıkarılır.

Ref	Frame 1	Frame 2	Frame 3	Fault?
1	1	–	–	F
2	1	2	–	F
3	1	2	3	F
4	4	2	3	F (1 en eski)
1	4	1	3	F (2 en eski)
2	4	1	2	F (3 en eski)
5	5	1	2	F (4 en eski)
1	5	1	2	– (hit)
2	5	1	2	– (hit)
3	3	1	2	F (5 en eski)
4	3	4	2	F (1 en eski)
5	3	4	5	F (2 en eski)

LRU toplam page fault = 10

Bu, **Belady'nin Anomalisi** değildir! Anomali: daha fazla frame ekleyince FIFO'nun daha kötü çalışmasıdır. Burada sadece FIFO, bu string için LRU'dan iyi çıktı – bu her zaman böyle olmaz.

(c) Optimal:

Optimal'de gelecekte en geç kullanılacak sayfa çıkarılır (implementasyonu imkânsız, benchmark).

Ref	Frame 1	Frame 2	Frame 3	Çıkarılan	Fault?
1	1	–	–	–	F
2	1	2	–	–	F
3	1	2	3	–	F
4	1	2	4	3 (çıkışı en geç)	F
1	1	2	4	–	– (hit)
2	1	2	4	–	– (hit)
5	1	2	5	4 (çıkışı en geç: 4 ref=10'da, 5 ref=11'de)	F
1	1	2	5	–	– (hit)
2	1	2	5	–	– (hit)
3	3	2	5	1 (çıkışı en geç ya da hiç gelmeyecek)	F
4	4	2	5	3 (artık gelmeyecek)	F
5	4	2	5	–	– (hit)

Optimal toplam page fault = 7

(d) Karşılaştırma:

Algoritma	Page Fault Sayısı
FIFO	9
LRU	10
Optimal	7

Optimal en az page fault ile en iyi performansı göstermektedir. Ancak Optimal, gelecekteki referanslara erişim gerektirdiğinden **gerçekte uygulanamaz** – yalnızca diğer algoritmaların ne kadar iyi çalıştığını **ölçmek** için benchmark olarak kullanılır.

Bu string için FIFO, LRU'dan iyi çıkmıştır. Bu sıra dışı bir durumdur – bu örnek, FIFO'nun **Belady's Anomaly**'e açık olduğunu da hatırlatmaktadır (daha fazla frame eklenince daha kötü çalışabilir, LRU'da bu olmaz).

2.2 Soru B2 – Farklı String ile FIFO + LRU (4 Frame)

Soru B2

4 page frame'i olan bir sistemde aşağıdaki reference string verilmiştir:

0 1 2 3 0 1 4 0 1 2 3 4

- (a) **FIFO** algoritmasını uygulayarak toplam page fault sayısını bulunuz.
- (b) **LRU** algoritmasını uygulayarak toplam page fault sayısını bulunuz.
- (c) Aynı string için 3 frame ile FIFO kaç fault verir? Bu, Belady'nin Anomalisi hakkında ne söyler?

CEVAP

(a) **FIFO, 4 Frame:**

Ref	F1	F2	F3	F4	Fault?
0	0	–	–	–	F
1	0	1	–	–	F
2	0	1	2	–	F
3	0	1	2	3	F
0	0	1	2	3	– (hit)
1	0	1	2	3	– (hit)
4	4	1	2	3	F (0 çıktı)
0	4	0	2	3	F (1 çıktı)
1	4	0	1	3	F (2 çıktı)
2	4	0	1	2	F (3 çıktı)
3	3	0	1	2	F (4 çıktı)
4	3	4	1	2	F (0 çıktı)

FIFO (4 frame): 10 page fault

(b) **LRU, 4 Frame:**

Ref	F1	F2	F3	F4	Fault?
0	0	–	–	–	F
1	0	1	–	–	F
2	0	1	2	–	F
3	0	1	2	3	F
0	0	1	2	3	– (hit)
1	0	1	2	3	– (hit)
4	0	1	4	3	F (2 en eski)
0	0	1	4	3	– (hit)
1	0	1	4	3	– (hit)
2	0	1	4	2	F (3 en eski)
3	0	1	3	2	F (4 en eski)
4	0	4	3	2	F (1 en eski)

LRU (4 frame): 8 page fault

(c) **Belady's Anomaly** gösteriminize göre 3 frame ile FIFO:

(Hesap kısaltılmış – çıktısına doğrulayın:)

3 frame ile FIFO bu string için **9 page fault** verir.

4 frame ile FIFO ise **10 page fault** vermektedir.

Belady'nin Anomalisi

3 frame → 9 fault, 4 frame → 10 fault!

Daha fazla frame eklenince FIFO daha **kötü** çalıştı. Bu, FIFO'nun **stack property**'yi sağlamadığını göstermektedir.

LRU ve Optimal stack property'yi sağlar ⇒ bunlarda Belady's Anomaly olamaz.

3 Kategori C: Kavramsal Virtual Memory Soruları

3.1 Soru C1 – Thrashing, Working Set, PFF

Soru C1

- (a) **Demand paging** nedir? Hangi problemi çözer?
- (b) **Thrashing** nedir? Neden oluşur ve nasıl önlenir?
- (c) **Working set** kavramını tanımlayın. Thrashing ile ilişkisini açıklayın.
- (d) **Page-Fault Frequency (PFF)** ne için kullanılır?

CEVAP

(a) **Demand Paging:**

Demand paging, bir sayfanın **yalnızca gerçekten erişildiğinde** belleğe yüklendiği mekanizmadır. Program başladığında tüm sayfalarını belleğe almak yerine, sayfa ilk kez erişildiğinde **page fault** tetiklenir ve o sayfa diske alınır. Bu sayede:

- Fiziksel bellekte yalnızca aktif sayfalar bulunur.
- Program başlangıcı hızlanır.
- Aynı fiziksel bellekle daha fazla süreç çalıştırılabilir (multiprogramming artar).

(b) **Thrashing:**

Bir sürecinin working set'ini bellekte tutmak için **yeterli page frame'i olmaması** durumudur. Bu durumda:

- Sürec sürekli page fault üretir.
- Her fault'ta bir sayfa diske yazılıp başka bir sayfa okunur.
- CPU zamanının büyük kısmı sayfa takas işlemlerine harcanır.

Önleme:

- Working set algoritması kullanmak.
- PFF'ye göre frame tahsisini dinamik ayarlamak.
- Yeterli frame olmayan süreci **suspend** etmek (swap out).

(c) Working Set:

$W(t, \Delta)$: t anında son Δ bellek referansında erişilen sayfalar kümesi. Δ **working set window**'dur.

Thrashing ile ilişkisi: Eğer bir sürecin working set'i tamamen bellekte tutulabiliyorsa thrashing olmaz. Tüm süreçlerin working set boyutlarının toplamı mevcut frame sayısını aşarsa thrashing kaçınılmaz.

(d) Page-Fault Frequency (PFF):

PFF, saniyedeki page fault oranını **moving average** olarak izler ve frame tahsisini dinamik yönetir:

- $PFF > \text{üst sınır} \Rightarrow$ Sürece ek frame ver.
- $PFF < \text{alt sınır} \Rightarrow$ Süreçten frame al.
- Verilecek ek frame yoksa $\Rightarrow$ Süreci suspend et.

3.2 Soru C2 – TLB, Page Fault Mekanizması, Karşılaştırma**Soru C2**

- (a) **TLB (Translation Lookaside Buffer)** nedir? **TLB hit** ve **TLB miss** durumlarını farklı bellek erişim sayısı açısından karşılaştırın.
- (b) Bir page fault oluştuğunda işlemin sistemi hangi adımları izler?
- (c) **Local** ve **global** page replacement policy arasındaki fark nedir? Hangisi genellikle daha iyidir ve neden?

CEVAP**(a) TLB:**

TLB, page-frame eşlemelerini **cache**'leyen hızlı donanım tamponu. Her adres çevirisi önce TLB'de aranır:

	TLB Hit	TLB Miss
TLB erişimi	1	1
Page table erişimi	0	1
Veri erişimi	1	1
Toplam	2	3

TLB, sık kullanılan eşlemeleri cache'leyerek her bellek erişiminde page table'a gitme zorunluluğunu ortadan kaldırır. Context switch'te genellikle **flush** edilir (bazı mimarilerde ASID ile önlür).

(b) Page Fault Mekanizması:

1. Donanım, geçersiz page table girdisi (valid bit = 0) tespit eder.
2. **Trap** oluşur; kontrol OS'a geçer.
3. OS, adresin gerçekten geçersiz mi yoksa bellekte olmayan geçerli bir sayfa mı olduğunu kontrol eder.
4. Geçersizse: süreci **sonlandır** (segmentation fault).
5. Geçerliyse: boş frame bul (gerekirse page replacement çalıştır).
6. İlgili sayfayı diskten frame'e yükle.
7. Page table'i güncelle (valid bit = 1, frame number yaz).

8. TLB güncelle.

9. Süreci kesmeden önce yapılan komutu **yeniden çalıştır** (restart).

(c) **Local vs. Global Policy:**

Local policy: Page fault olan sürecin yalnızca kendi frame'lerinden birini çıkarır. Diğer süreçler etkilenmez.

Global policy: Herhangi bir süreçten frame alınabilir.

Global policy genellikle daha iyidir çünkü:

- Local policy'de working set büyürse thrashing kaçınılmaz; başka süreçten frame alınamaz.
- Global policy frame talebini dinamik yeniden dağıtabilir.

Önemli: Working Set ve WSClock **yalnızca local policy** ile çalışır. Sistem için "tüm süreçlerin working set'i" kavramı yoktur.

4 Hızlı Başvuru: Özet Tablo

Konu	Anahtar Nokta
Address translation	$VA = \text{page\#} \parallel \text{offset}$; $PA = \text{frame\#} \times \text{page_size} + \text{offset}$
Offset bit sayısı	$k = \log_2(\text{page size})$
TLB hit	2 bellek erişimi (TLB + veri)
TLB miss	3 bellek erişimi (TLB + page table + veri)
Page fault	Valid bit = 0; OS disk'ten sayfayı yükler, komutu restart eder
FIFO	En eski sayfa çıkar; Belady's Anomaly mümkün
LRU	En uzun süredir kullanılmayan çıkar; stack algorithm (Belady yok)
Optimal	Gelecekte en geç kullanılacak çıkar; implemente edilemez, benchmark
Thrashing	Working set > mevcut frame $\Rightarrow$ sürekli swap
Working set	$W(t, \Delta)$: son Δ ref'teki sayfalar
PFF	Moving average; yükseğe frame ekle, düşüğe al
Local vs. Global	Global genelde daha iyi; Working Set yalnızca local

Son Notlar:

Address translation sorularında **binary'e çevirme adımını atlama** en yaygın hatadır.

Offset bit sayısını **her zaman ilk hesapla**, geri kalan biti page number yap.

FIFO tablosunda **kimin çıktığını** yazmayı unutma — parça puan orada.